

DeepSeek Harness

多设备 Agent 与 Mobile / Physical Capability 架构

从 PC-centric Harness、Tailscale 私有设备网络到跨设备 Capability Fabric 与 Situated / Physical AI

1 摘要

DeepSeek Harness (DSH) 的可扩展对象从单机 Agent runtime 延伸为长期存在的跨设备逻辑 Agent。当前阶段以 PC/WSL 作为 canonical execution node, 承担 LLM 请求、Agent Loop、session/context、workspace、plugin runtime 与绝大多数 Tool; Tailscale 提供私有 device fabric; 手机先作为同一 Web runtime 的移动 surface, 再通过轻量 capability service 暴露相机、位置、文件、Intent、通知、麦克风与传感器等 Android-native capability。随着更多节点加入, Tool surface 逐步演化为带有位置、权限、时延、带宽、功耗、隐私与用户在场约束的 distributed capability graph。Agent 的逻辑身份与任一具体设备解耦, 设备成为可发现、可授权、可失效和可替换的 execution / sensing / actuation node。

当前工程不变量: One canonical session truth • One workspace • PC-first orchestration • Many human surfaces • Many capability endpoints。 **长期演化不变量:** One logical agent • Many execution nodes • Capability- and context-aware placement。PC 权威是阶段性收敛策略, 不写死为系统最终形态。

Runtime 集中 当前模型调用、context governance、Tool 调度和多数计算保留在 PC, 复用已有 DSH 定制与高性能执行环境。

Capability 分布 手机与后续边缘设备只暴露其本地独有能力; 设备能力通过 manifest 进入统一 Tool graph。

Surface 投影 Desktop、PWA、WebView、Voice、Tablet 等界面是同一 logical runtime 的不同 projection, 避免为每个设备复制 application state。

Placement 显式化 computation 与 data 的移动由 capability location、network、privacy、energy、authority 与 user-presence 共同决定。

2 运行时基线

PC 端 DSH 已形成较深的 session/context governance 定制。leavelet/deepseek-harness 提供有界长会话读取、packed retention、上下文检查与区间选择、历史召回、按模型容量规划压缩和可恢复摘要审阅; MuWinds/dsh-archived-sessions 增加归档会话列出、预览、释放、删除与容量统计; xiaobright/dsh-anchored-standard 引入受控首轮 Tool surface、上下文门控、wire-think 路由、压缩感知阶段提升、session prefab 播种、跨平台 shell path 与稳健指令发现。

完整 Android Harness 会复制 session state、context pipeline、plugin lifecycle、Tool runtime 与升级路径, 并把同一套深度定制长期分叉到两个平台。移动端当前提供的增量主要来自移动交互、内容采集和 Android OS capability, 因此 PC 保持主运行节点, 手机保持薄 surface 与 capability endpoint。

3 演化脉络: DSH → Tailscale → Mobile / PC → Physical AI

DSH 的关键资产并非聊天 UI, 而是把 model、context、Tool、permission、execution、session lifecycle 与 Human control 组织成一个可持续运行的 harness。Tailscale 改变了该 harness 的空间边界: PC 上的 runtime 不再只属于桌面的单次使用, 而可以在私有网络中被手机、平板、家中节点与后续边缘设备持续访问。手机接入后, 第一步只是远程 surface; 当 camera、microphone、location、notification、Intent、BLE 与 sensor 被注册为 Tool, 手机开始成为 Agent 的 sensing / actuation endpoint。

DSH runtime → private device fabric (Tailscale) → mobile control / capture surface → mobile-native capability endpoint → multi-node capability graph → context- and network-aware execution placement → situated / physical agent

这里的 Physical AI 不以机械臂或灵巧手作为必要前提。只要 Agent 的 observation、decision 与 action 开始依赖真实设备、物理位置、网络状态、传感器、用户在场性和外部副作用, 它已经进入更广义的 situated / physical execution problem。机器人控制可以成为后续节点类型之一, 和手机、PC、服务器、摄像头、BLE 设备共享同一 capability / permission / context 语义。

通信也由透明 pipe 进入 Agent policy。相同的视觉任务可能在手机端本地处理、上传原始图片到 PC、只上传压缩 observation、等待更稳定网络后执行, 或因隐私策略完全禁止离开设备。where to execute、what to move 与 what to expose 成为 Tool routing 的一部分, 而不是部署阶段的静态配置。

3.1 Logical Agent 与设备节点分离

Agent identity、session continuity 与任务语义不绑定到某一进程或设备。当前 canonical state 位于 PC, 但 Tool schema 与 capability protocol 不把 pc 写成不可替代前提。后续 server、tablet、home machine、edge board 或机器人控制器均可注册为 node; PC 可以继续拥有最高 capability density, 也可以在特定任务中只承担 coordinator。

Capability descriptor 需要逐步容纳执行位置与物理约束:

```
CapabilityDescriptor { capability_id, node_id, schema_version, modality, availability, authority,
user_presence, latency_hint, bandwidth_hint, energy_hint, privacy_class, concurrency_class,
side_effect_class }
```

这些字段让 Agent runtime 能区分“某个 Tool 是否存在”与“此刻在哪里、以什么代价、在什么授权条件下可执行”。

4 当前目标与边界

目标	设计含义
私有远程控制	手机在 5G、校园网或异地 Wi-Fi 下通过 tailnet 访问 PC DSH；DSH 无需暴露公网端口、DDNS 或公共反向代理。
Canonical state	session、context、archive、workspace、Agent state 和 Tool result commit 当前以 PC runtime 为准；移动端不维护第二份 authoritative state。
移动能力进入 Tool graph	PC Agent 可调用手机 camera、location、file picker、Intent、microphone 等 capability；schema、调度与结果归档仍由统一 ToolRuntime 管理。
UI 与 runtime 解耦	PWA/WebView 加载同一 DSH Web application；移动端只在 Web API 覆盖不足时增加 native bridge。
可扩展 execution placement	协议层保留 device / location / authority / resource 元数据，为后续 tablet、server、edge node 与 physical device 预留 placement 语义。

首版不承担独立离线移动 Agent、双端 session replication、公开多租户服务与依赖 root 的系统级控制。独立移动运行时、peer-to-peer state replication 与无人值守 physical actuation 属于后续架构分支。

5 总体架构

系统分为 Control Plane、Compute Plane、Capability Plane 与 Device Fabric。Control Plane 接收 Desktop、Phone、未来 Voice/Tablet surface 的 Human 指令；Compute Plane 承担 model call、Agent Loop、session/context 与计算密集型 Tool；Capability Plane 暴露设备本地 sensing / actuation；Tailscale 提供私有 reachability 与设备身份基础。四层之间的边界由 DSH 的 session、Tool、permission 与 audit semantics 统一约束。

```
Control Plane Human → any surface → logical agent Compute Plane node → LLM / shell / files /
jobs / heavy compute Capability Plane node → local sensing / OS action / side effect Device Fabric
Tailscale → private reachability + peer identity
```

5.1 PC 主运行节点

PC 持有当前 canonical session store、context pipeline、workspace、Agent Loop 与 ToolRuntime。ExecutionRouter 位于 ToolRuntime 与 executor 之间：PC-local Tool 进入本机 executor；device-local Tool 依据 capability descriptor 路由至对应节点。结果回到 canonical event/history 后进入 replay、compression、archive 与 observability。

5.2 Mobile Surface 与 Capability Service

移动侧由 Web/PWA surface 与 Native Capability Service 组成。前者直接加载 DSH Web application；后者补足 Web 平台无法稳定或安全完成的 Share Target、相机、位置、Android Intent、文件选择、后台服务、通知、麦克风和后续 sensor/BLE 能力。两部分共享 device identity 与本地 permission state，不复制 LLM adapter、Agent Loop、context manager 或通用 plugin runtime。

5.3 Surface 是 projection，不是第二个 application

Desktop WebUI、手机 PWA、未来 voice interface 与其他界面读取和操纵的是同一 runtime state。UI 差异来自 projection 与 device affordance，而不是一份额外业务状态。手机端无需与 PC “同步同一套产品”；两端本来就只是同一个 logical environment 的不同观察与控制入口。

6 Tool 执行语义

手机本地操作在主 runtime 完成 Tool lookup、schema validation、调度、权限需求判定与 execution target 选择。设备只接收结构化 invocation，不重复解析自然语言。PC-local 与 device-local Tool 因而能够进入同一 task graph，而设备端保持最小语义面。

```
ToolInvocation { call_id, session_id, target_node, capability, args, deadline, approval_policy,
idempotency_key } ToolResult { call_id, status, data | error, node_state, timing, audit_metadata }
```

路由粒度落在 capability，而不是整个 Agent。“手机模式 / PC 模式”只适合作为 UI 偏好；执行系统应按 Tool、node、resource 与 policy 逐次决定 placement。单个任务可以连续调用 phone.camera.capture、pc.vision.*、pc.web.fetch、server.gpu.* 与 pc.files.write。

6.1 文件与内容迁移

普通手机文件采用“迁移 artifact，保留 computation”的路径。Share Target 或文件选择器把 PDF、图片、URL、文本写入 canonical workspace 或 session attachment 后，解析、检索、转换、代码生成与归档继续在高能力节点完成。对连续流、隐私数据或低带宽场景则允许移动 computation 而非原始 data。

6.2 动态 Capability Manifest

设备上线后注册动态 manifest。每项能力携带 schema version、availability、OS permission、approval requirement、foreground requirement、concurrency class、side-effect class 与可选 resource hints。权限撤销、设备锁定、App 退到后台、网络恶化或硬件不可用都会改变 capability surface。

Capability	典型状态	执行约束
phone.camera.capture	available / user-presence-required	相机按设备串行；可要求前台 UI；大图上传受 bandwidth/privacy policy 影响。
phone.microphone.sample	foreground / consent-required	录音、持续采集与一次性语音输入使用不同 grant 和 lifecycle。
phone.location.get	granted / denied / foreground-only	精度、后台可用性、缓存时效与 OS permission 绑定。
phone.file.pick	available	系统选择器授予单次文档访问；随后可迁移到 canonical workspace。
phone.intent.open	approval-required	产生跨 App 或外部副作用；记录 target 与审批。
edge.sensor.*	online / stale / unavailable	未来节点；observation 需要 timestamp、freshness 与 node provenance。

7 移动交互模型

手机承担 Control Surface、Capture Surface 与 Presentation Surface。Control Surface 继续 session、发送 prompt、处理 approval、观察 job 与 interrupt/resume；Capture Surface 通过 Share Sheet、文件、camera、microphone 等把当下环境送入 Agent；Presentation Surface 复用 PC 托管的 Web runtime，包括 sidebar、session inspector 与后续工作台插件。

交互边界： APK/PWA 的价值是把移动设备的 affordance 接入同一 logical environment。UI 变化优先留在 Web runtime；只有 capability contract 或 Android-native API 发生变化时才需要升级 native layer。

收到的 artifact 进入统一 ingest pipeline：识别 MIME/URL → 建立 provenance → 写入 workspace 或 temporary attachment → 绑定 session/task → 触发 Agent execution。网页、GitHub 链接、论文 PDF、相册图片、选中文本与语音片段共享同一入口。

8 网络、权限与物理约束

Tailscale 作为当前 device fabric。Android 客户端支持直接加入 tailnet [10]；tailscale serve 可把 PC 上仅监听 localhost 的 DSH Web 服务私有暴露到 tailnet，Funnel 才进入公网路径 [11]。连接优先设备间直连，无法直连时回退到 relay；WireGuard 端到端加密保持数据平面私有 [12]。

网络可达性只完成 peer reachability。完整权限链包括：

1. **Tailnet policy.** grants 限定 phone → PC Web/API 与 PC → phone capability RPC 的方向、目标和端口 [13]。
2. **Device / application identity.** pairing 绑定稳定 node identity，RPC 携带 device/session/call identity，未知 peer 不进入 capability plane。
3. **OS permission.** camera/location/microphone/notification 等系统权限直接决定 capability availability。
4. **DSH capability grant.** 低风险能力可 once/session/always 授权；发送消息、录音、外部写入、通知回复与 actuator control 保留更强审批与 audit。
5. **Context policy.** 原始传感器数据、位置、屏幕与个人文件可以标注 local-only、derived-only、shareable 等 privacy class，限制 data migration。

随着节点增多，network state 会进入 Agent decision：latency、bandwidth、packet loss、device battery、thermal state、privacy 与 user presence 都可能改变执行位置。通信与计算由静态部署关系转成 runtime placement problem。

9 性能与生命周期

当前模型请求、streaming parser、context management 与 PC-local Tool 原位执行；Tailscale RTT 主要作用于移动 UI event stream 与 device-native RPC。模型侧 TTFT 与 tokens/s 仍由 provider、网络与 PC runtime 主导。WebSocket/SSE 保持长连接；大文件一次迁移到高能力节点；camera/microphone/screen 等独占资源具有设备级并发约束。

长期存在的 Agent 还需要明确 continuity：进程重启、设备掉线、PC 休眠或 execution node 迁移后，session identity、task state、pending side effect 与 capability lease 如何恢复。当前以 canonical PC event log 解决大部分连续性问题；未来进入 replicated state 时，需要重新区分 durable shared state、local ephemeral state 与 node-local execution state。

9.1 失效与恢复

故障	处理机制
手机离线 / 网络切换	heartbeat/lease 使 capability 转为 unavailable；重连后重新注册 manifest，不自动重放未知副作用。
RPC 超时或重复投递	call_id + idempotency_key + deadline；副作用 Tool 记录执行状态并拒绝重复提交。
权限运行时撤销	设备更新 manifest；在途调用返回结构化 permission error，并写入原 Tool result。
Android App 被系统回收	需要持续任务的 capability 使用受控 foreground/background service；presentation lifecycle 与 capability service 分离。
Canonical PC 不可用	当前阶段 logical agent 暂停；后续可通过 replicated state / alternate coordinator 扩展，但不伪装成现有能力。
Sensor observation 过期	observation 携带 timestamp/freshness/provenance；Agent 不把 stale physical context 当作当前事实。

10 既有 DSH / Android 路径

社区项目分别验证远程移动 UI、原生 Android client、本地 Android runtime、PRoot compatibility layer 与 ADB device control。差异集中在 runtime authority、移动 capability 深度与维护边界。

路径	执行权威	移动 UI	手机 capability	状态 / 维护特征	可复用部分
DSH Multi-Device	PC-first	WebUI + native bridge	一等能力；RPC + permission broker	一套 session/context/plugin runtime；设备注册 capability	远程控制、PC 强环境、手机 OS 能力与后续多节点扩展
saya-ch/dsh-mobile [4]	PC DSH	同一 Web origin/WebView	主要为移动访问与 UI	不修改 DSH；同一 session/workspace	presentation / remote-access baseline
Hakunm/dsh-android-app [5]	PC DSH	原生 Android client	聊天、审批、文件、模型管理	远程 API 维持同一 workspace/session/files	native client 与 remote management
fengnanrui/DSH-Android [6]	手机本地	原生 Android	本机实现	复制 session/Agent/plugin 语义；独立维护	standalone/offline 分支
WSK-build/DSHBox [7]	手机本地 DSH	本机 WebUI	Android 能力仍需 bridge	PRoot Debian + Node + DSH	upstream runtime compatibility
ZSeven-W/dsh-android [8]	PC DSH	DSH 内 device panel	ADB 深度控制 Android	Agent tools + ADB serial identity	PC Agent → Android device 的 capability 路径
0xcafff/codex-web [9]	Host Codex	Desktop UI → Browser	无手机 OS bridge	Electron IPC → WebSocket shim	runtime/client lifecycle separation

dsh-mobile 与 codex-web 支持 runtime / presentation 分离；dsh-android 支持 PC Agent 把 Android 纳入 Tool graph；本地 Android Harness 证明完整 runtime 可以迁移到手机，但同时暴露双运行时维护成本。当前主线因此保持 PC-first authority，同时把 protocol 设计成可跨 node 扩展。

11 相邻技术生态

这些社区处理的对象和 DSH 不完全相同，但共享真实的系统接口：runtime state、program representation、persistent/shared state、capability、live execution、debugging、permission、direct manipulation 与 multi-device environment。它们适合作为继续寻找新 primitive 的长期入口。

11.1 Malleable Systems Collective

Malleable Systems Collective Catalog 聚集 malleable software、end-user programming、personal computing、interactive systems 与 alternative programming environments。Catalog 本身把项目、论文、作者和讨论混在同一张关系图中，适合沿系统与作者继续爬，而不是只读一个固定领域综述 [14]。

对 DSH 最直接的几个对象：

- Glamorous Toolkit：moldable development。domain object 可以拥有 contextual view、search、action、inspector 与 debugger；Job、AgentRun、Queue、ToolInvocation 可以被理解成自带专用观察与操作界面的运行时对象 [15]。
- Webstrates：把网页 DOM 变成持久、协作、可同步的应用 substrate，改变 document、application 与 shared state 的边界 [16]。
- Folk Computer：reactive database、projection mapping、physical objects 与 declarative coordination。纸片、摄像头、投影与 statement dependency 直接进入 programming environment，是从 software surface 走向 physical computing 的具体实现 [17]。

这里能看到的“好玩东西”通常是新的 object model 与 programming medium: 对象自己长出 inspector; 网页从 UI 变成 persistent state; 真实桌面上的纸片进入 reactive database。它们对 DSH 的价值不在复刻某个产品, 而在重新考虑 Agent runtime 是否也能成为可塑、可投影、可直接操纵的环境。

11.2 Future of Coding / Feeling of Computing

Future of Coding Meetups 与 2026 Weekly 持续汇集个人 prototype、alternative programming environments、visual / live programming、small languages、notebook、direct manipulation 与 personal computing 实验 [18]。社区的信息增量来自大量尚未形成稳定领域名称的个人 hack, Two Minute Week、Our Work、meetup demo 与 discussion thread 往往比成熟论文更早暴露新 primitive。

2026 年的具体例子已经和多设备 DSH 发生直接连接。Ella Hoepfner 展示了以嵌套圆形 AST 代替文本 source 的 graphics programming environment, shader 实时重编译, 数值表达式可以直接变成 slider; source、GUI control 与 AST 是同一 underlying representation 的不同 projection [19]。Tom Larkworthy 的 Lopebook / Lopecode 则把 notebook runtime 序列化成为可嵌入、可搬运的单体环境, 甚至能在另一个 host 页面 console 中重新载入并继续编辑; 它把 portable runtime state、self-editing environment 与 host integration 变成可直接玩的系统对象 [20]。同一周的社区讨论还直接质疑“一人、一机、一个 isolated environment”的 personal-computing 假设, 并提出跨多个 peer 的 single, shared, persistent programming environment, 用于 multi-device 与 social setting [20]。

这里值得追的对象是 projectional editor、portable runtime、shared persistent environment、embedded notebook 与 alternative source representation。对于 DSH, 它们对应一个更深的 UI / runtime 问题: PWA、Desktop、Voice 与未来 physical surface 是否只是同一 runtime object graph 的不同 projection, 而不是多个客户端产品。

11.3 LIVE Workshop

LIVE 长期讨论 live programming、structure-aware editing、programming by example、partial program、direct manipulation 与 execution visualization [21]; 历届 Program 可以从具体 demo / paper 继续追作者与系统 [22]。

这里能看到 typed holes、live execution、visible intermediate state、direct manipulation of program output 等技术。历史 program 中的 PANE 把 concrete intermediate data 直接暴露到编程过程; Hazel 让带 hole 的不完整程序继续保持类型意义; programming-by-example 与 SVG intermediate manipulation 则让用户直接改结果, 再由系统反推出程序变化 [22]。对 Agent tooling 的迁移点很直接: 模型经常产生 partial program、partial workflow 与未决 Tool arguments; typed hole / partial state 可以让 runtime 保留“已知结构 + 未完成位置 + constraint”, 而不是把不完整 JSON、代码或计划当成失败字符串。Live execution 还可以把 prompt / permission / memory policy 的修改直接映射到一组运行中的 trace 与 evaluation case。

11.4 Ink & Switch

Ink & Switch Projects 长期制作 local-first software、malleable software、programmable ink、collaborative tools 与 universal version-control prototype, 常以 essay、lab notebook 和实现日志公开设计过程 [23]。

几个与 DSH 当前问题相连的项目:

- Potluck: 从自由文本逐步增加 search、formula 与 dynamic annotation, 把 document 渐进式长成 personal software; 研究笔记、实验记录和 Agent artifact 可以沿相同方式从文本逐步长出可执行操作 [24]。
- Patchwork: 把 branching、diff、history 从 Git / Word / Figma 的局部功能抽成跨 artifact 的 variation substrate; Agent 生成的多个网页、文档或方案可以进入 lineage / diff / merge graph, 而不是散落成互不相干的文件 [23]。
- Patchwork Local-First Task Framework 与 implementation notebook: task、worker、queue、success/failure、log 与 timing history 被放进 local-first shared substrate; 与 DSH 的 job、worker、queue、side effect 和离线执行已经处在同一个系统问题空间 [25]。
- Livelymerge notebook: 尝试把 live object heap 放入 persistent shared document。Convergence Is Not Enough 直接展示 CRDT 最终收敛仍可能破坏 application invariant; Local State 又暴露 local ephemeral state 与 shared durable state 的边界, 一个用户打开的操作 halo 不应该被同步并永久保存给其他人 [26]。DSH 同样需要区分 session memory、device-local UI state、sensor observation、tool execution state 与 durable shared context。

这里最有价值的是看到“漂亮 primitive 落地后怎么坏掉”: convergence 不能替代 invariant, shared heap 会误吞 local UI state, task worker 需要处理 side effect、ownership 和 offline completion。它提供的是可迁移的 failure mode, 而不只是 local-first 口号。

11.5 PLATEAU

PLATEAU 2026 位于 Programming Languages 与 HCI 交界, program 直接把 specification、execution visualization、types + prompts、permissions、question-oriented tools、debugging 与 direct manipulation 放在同一空间 [27]。

2026 program 中值得继续追的对象包括 Synthesizing Visual Specifications、Tracers for debugging and program exploration、Towards Typed Conversational Interfaces、Security Types for Usable Permissions Prompts、Aporia: Asking Questions to Elicit User Decisions 与 Explorable Theorems [27]。这些题目分别连接 DSH 的 artifact invariant、Agent trace、conversation protocol、capability permission、人类 decision boundary 与可交互 assumption space。

PLATEAU 的价值在于把“Agent UI”拆成更精确的技术对象：conversation 可以具有 type；permission prompt 可以进入 type/security 设计；trace 可以按照 causal dependency 和 state mutation 重组；用户问题可以被当作显式 decision point，而不是模型自由文本里的偶然追问。

12 接口与组件

12.1 PC 端 DSH Plugin

- DeviceRegistry: device identity、在线状态、manifest version、capability descriptor 与 heartbeat lease。
- ExecutionRouter: 依据 Tool descriptor、device availability、policy、network/resource hints 与 side-effect class 选择 executor。
- MobileRpcTransport: tailnet 上的认证双向 RPC/stream，支持 cancellation、deadline、reconnect、backpressure 与 idempotency。
- CapabilityToolProjection: 把动态 manifest 投影为 Agent 可见 Tool；不可用 capability 从 surface 移除或显式标记。
- ContextPlacementPolicy: 决定 raw data、derived observation 与 execution 在 node 间如何移动。
- AuditLog: 关联 invocation、approval、node、result、timing、context provenance 与 session event。

12.2 Android Client

- Web / PWA shell: 加载 tailnet 内 DSH WebUI，处理安全 origin、登录/配对、深链与文件上传。
- Capability Service: 接收结构化 invocation、维护 manifest、调用 Android adapter、返回结构化结果。
- Permission Broker: 统一 OS permission、应用级 grant、Human approval 与风险级别。
- Share Receiver: 接收 ACTION_SEND / ACTION_SEND_MULTIPLE 并进入统一 ingest pipeline。
- Lifecycle Service: 仅为持续 capability 保持 foreground/background execution，避免把 UI Activity 误当 runtime。

13 实施路径

Phase 0 — Network baseline	PC 保持 DSH 仅监听 localhost，通过 Tailscale Serve 或 tailnet IP 暴露 WebUI；验证 session continuity、streaming、job reconnect 与移动网络访问。
Phase 1 — PWA surface	优先使用 Web/PWA 完成移动访问、session continuation、approval、Share Target 与基础 capture；不提前制造完整 Android application。
Phase 2 — Thin native bridge	只为 Web API 无法覆盖的能力增加 Android capability service: file.pick/upload、camera.capture、location.get、microphone.sample、intent.open。
Phase 3 — Capability protocol	实现 registry、manifest、RPC envelope、permission broker、provenance、timeout/cancel/idempotency 与 Tool-level target selection。
Phase 4 — Multi-node placement	加入 server/tablet/home machine；Tool descriptor 增加 location/network/privacy/resource hints，开始记录 placement decision 与实际性能。
Phase 5 — Ambient / physical capability	扩展 notification、sensor、BLE、camera stream 与其他 edge node；建立 freshness、user-presence、continuous permission 与 physical side-effect policy。
Phase 6 — Persistent distributed agent	评估 canonical PC authority 的边界；只有在真实需求出现后再引入 replicated shared state、alternate coordinator 与跨节点 continuity。

14 验证指标

维度	验证问题	测量
响应性能	移动 surface 是否显著增加 token/event 可见延迟？	PC local 与 phone-over-tailnet 的 TTFT presentation delta、event RTT；direct / relay 分开统计。
Tool latency	device-native Tool 的网络与 permission 成本是否可接受？	camera/location/intent invocation → result wall-clock，拆分 network/approval/OS API。
Placement	data move 与 computation move 哪种更优？	不同 bandwidth/latency/privacy 场景下记录 execution node、bytes transferred、wall-clock 与 failure。
状态一致性	断网、重连、重复 RPC 是否产生重复副作用或 session divergence？	fault injection: drop/reorder/retry/reconnect，检查 call_id、event log 与 side-effect state。
权限正确性	网络可达时能否继续拒绝未授权 capability 与 context？	grant matrix、OS permission revoke、锁屏、后台、unknown device、local-only data 测试。

维度	验证问题	测量
连续性	进程/设备变化后 logical agent 是否保持可恢复任务语义？	restart/sleep/node-loss 测试 pending task、approval、lease 与 event replay。
维护成本	PC runtime 更新后移动端是否保持可用？	WebUI/plugin 更新回归；仅 capability contract 变化时升级 native layer。

15 风险与开放问题

Android lifecycle、后台 execution、通知访问、录音、屏幕与跨 App 操作受系统权限和发行渠道政策约束。远程副作用需要 idempotency、audit 与 Human approval；read Tool 可透明重试，外部写入与 physical actuation 不能沿用同一恢复语义。Web runtime 与 native bridge 之间只保留小而稳定的 capability protocol，避免 UI 更新变成 APK 同步工程。

更长期的风险来自抽象边界本身。把所有 device state 都塞进 shared context 会形成隐私、带宽和 stale-state 问题；把所有执行都中心化会丢失 latency / offline / physical locality；过早做完整 distributed state 又会引入 replication、consistency 与 debugging 复杂度。PC-first + explicit capability location 提供一条渐进路径：先扩展 Agent 可调用世界，再由实际 failure mode 决定哪些 state 和 computation 值得真正分布式化。

16 架构决策

ARCHITECTURE DECISION

近期采用 “PC-first DSH Runtime + Tailscale Device Fabric + PWA / Thin Native Capability Bridge”；协议层面按跨设备 logical agent 设计。

PC 继续承担 canonical session/context、workspace、model call、plugin runtime 和多数 Tool；手机先复用同一 Web runtime，再暴露 Android-native capability。Tool routing 按 capability / node 细分，Human 可从任意 surface 发起任务。node_id、authority、privacy、resource 与 network metadata 从首版进入协议，使后续 tablet、server、edge sensor 与 physical actuator 可以加入同一 capability graph，而无需重新定义 Agent / Tool 基础语义。

17 参考资料

1. **leavelet/deepseek-harness**. <https://github.com/leavelet/deepseek-harness>
2. **MuWinds/dsh-archived-sessions**. <https://github.com/MuWinds/dsh-archived-sessions>
3. **xiaobright/dsh-anchored-standard**. <https://github.com/xiaobright/dsh-anchored-standard>
4. **saya-ch/dsh-mobile**. <https://github.com/saya-ch/dsh-mobile>
5. **Hakunm/dsh-android-app**. <https://github.com/Hakunm/dsh-android-app>
6. **fengnanrui/DSH-Android**. <https://github.com/fengnanrui/DSH-Android>
7. **WSK-build/DSHBox**. <https://github.com/WSK-build/DSHBox>
8. **ZSeven-W/dsh-android**. <https://github.com/ZSeven-W/dsh-android>
9. **Oxcalf/codex-web**. <https://github.com/Oxcalf/codex-web>
10. **Tailscale — Install on Android**. <https://tailscale.com/docs/install/android>
11. **Tailscale Serve**. <https://tailscale.com/docs/features/tailscale-serve>
12. **Tailscale — Connection types**. <https://tailscale.com/docs/reference/connection-types>
13. **Tailscale — Grants**. <https://tailscale.com/docs/features/access-control/grants>
14. **Malleable Systems Collective — Catalog**. <https://malleable.systems/catalog/>
15. **Glamorous Toolkit — Learn the basics**. <https://book.gtoolkit.com/learn-the-basics-of-glamorous-toolkit-5hetr3qaqcfv42xap3v4j39o2>
16. **Webstrates**. <https://github.com/Webstrates/Webstrates>
17. **Folk Computer — Reactive database internals**. <https://folk.computer/notes/internals/db>
18. **Future of Coding — Meetups / Weekly 2026**. <https://futureofcoding.org/meetups> ; <https://newsletter.futureofcoding.org/2026/>
19. **Future of Coding Weekly 2026/01 Week 2 — Ella Hoepfner graphics environment**. <https://newsletter.futureofcoding.org/posts/future-of-coding-weekly-202601-week-2/>
20. **Future of Coding Weekly 2026/02 Week 2 — Lopebook / multi-peer persistent environment**. <https://newsletter.futureofcoding.org/posts/future-of-coding-weekly-202602-week-2/>
21. **LIVE — Workshop on Live Programming**. <https://liveprog.org/>
22. **LIVE — All Past Programs / 2018 notes**. <https://liveprog.org/all-programs.html> ; <https://futureofcoding.org/notes/live/2018.html>
23. **Ink & Switch — Projects / Patchwork**. <https://www.inkandswitch.com/project/> ; <https://www.inkandswitch.com/project/patchwork/>
24. **Ink & Switch — Potluck**. <https://www.inkandswitch.com/potluck/>
25. **Ink & Switch — Patchwork Task Framework**. <https://www.inkandswitch.com/patchwork/notebook/tasks-01/> ; <https://www.inkandswitch.com/patchwork/notebook/tasks-02/>
26. **Ink & Switch — Livelymerge**. <https://www.inkandswitch.com/livelymerge/notebook/> ; <https://www.inkandswitch.com/livelymerge/notebook/lm-02/> ; <https://www.inkandswitch.com/livelymerge/notebook/lm-04/>
27. **PLATEAU 2026 Program**. <https://2026.plateau-workshop.org/program>